Modeling and Forecasting Google Search Interest for BTS Using Time Series Methods

Authors: Caspar Leung and Elizabeth Louie

1. Abstract

This project analyzes and forecasts Google search interest for the K-pop group BTS using a variety of time series methods. Monthly Google Trends data from the past five years was examined to analyze the trend, seasonal, and stochastic components. After stabilizing variance using a lambda-transformation, the seasonal component was first estimated and removed to isolate the underlying trend. Two trend estimation approaches were then applied including a single polynomial fit over the full series and a piecewise polynomial fit separating early and late periods. The remaining residual processes were modeled using an ARMA process. To evaluate the influence of an extreme observation, forecasting was performed both with and without the January 2026 outlier. The results indicate that while the outlier temporarily raises forecast levels, the overall downward trajectory of BTS search interest remains largely unchanged.

2. Introduction

In this project we use time series methods to investigate a public search interest dataset for K-Pop group BTS. Our main goal is to identify ARMA models that best capture this time series data from the past five years in order to produce accurate forecasts for future publicity trends. K-pop popularity typically spreads through social media and online fandom networks, therefore examination of digital engagement through Google search trends provides a useful indicator of patterns over time. Further understanding of how BTS interest evolves over time has meaningful implications for cultural analysis for the music industry and the broader stories of celebrity lifetime dynamics.

Our research question for this data analysis asks how influential is the January 2026 outlier for estimating the trend component and ARMA forecasts of BTS search interest? To investigate this, we conducted separate trend and forecast analyses. The original time series dataset shown in Figure 1 presents two distinct interest intervals, one early high-interest phase and a recent decline phase with a gradual downward trend. To account for these two phases, we compare two trend estimation approaches: a single polynomial fit over the entire series and a piecewise polynomial model separating the early high-interest period from the later declining period. Secondly, January 2026 reveals an extreme outlier which raises the questions of genuine spike in BTS interest. We assessed this legitimacy by computing two forecast analyses with and without the outliers inclusion.

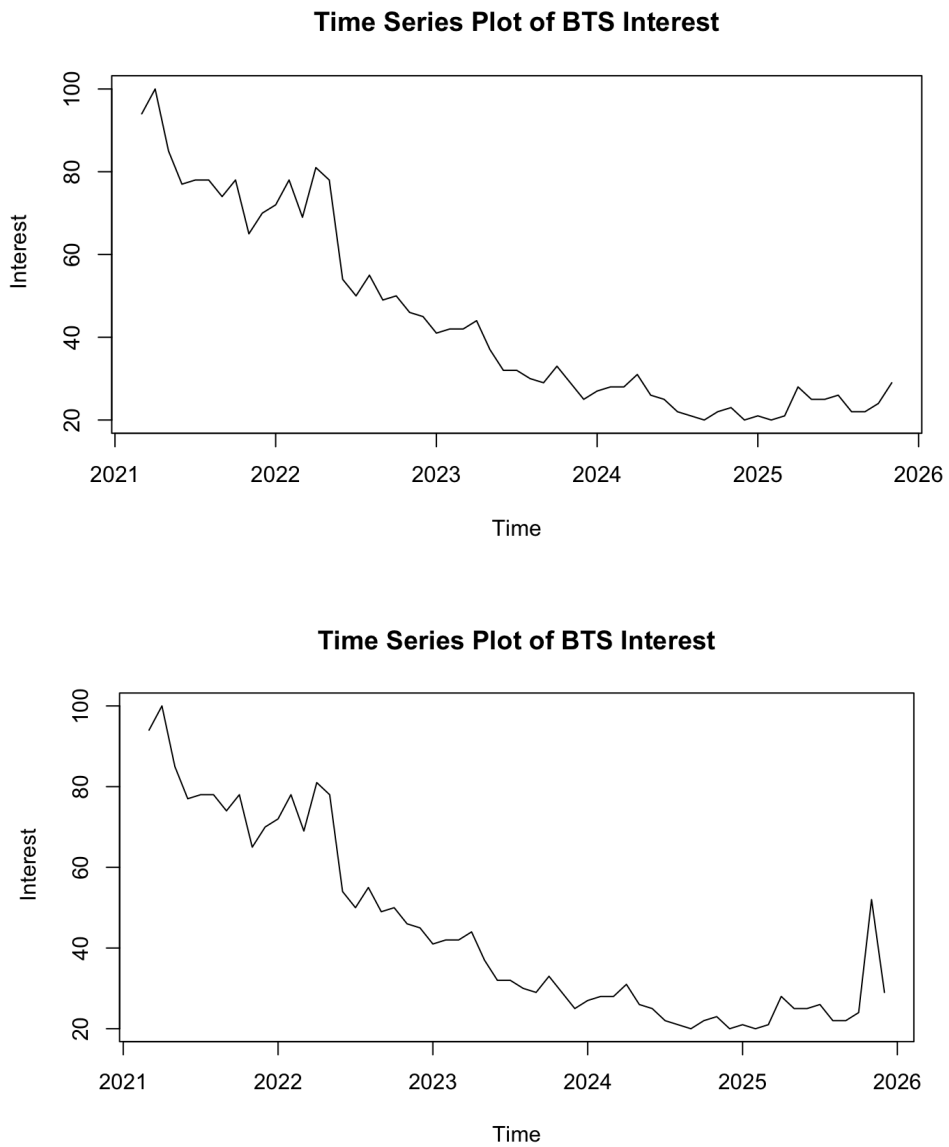

Figure 1: Time Series Plot of BTS Interest Mar 2021- Feb 2026 (top: outlier included, bottom: outlier removed)

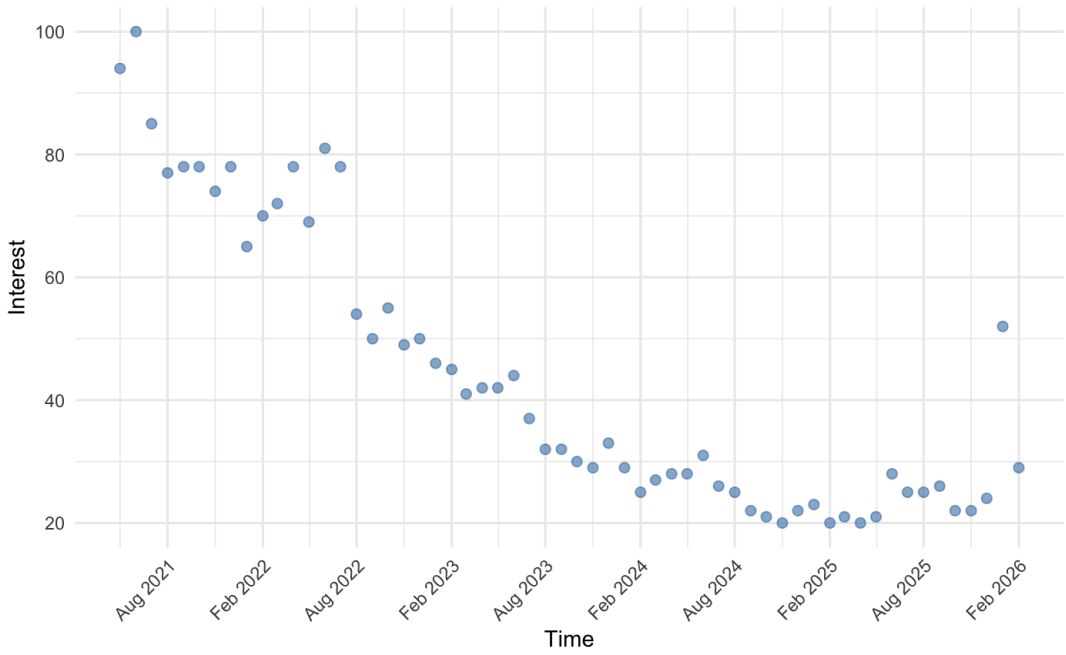

Figure 2: Scatter Plot of BTS Google Search Interest Mar 2021- Feb 2026

3. Data description

The data consists of 60 monthly observations of Google Trends search interest values for the keyword "BTS" from March 1, 2021 to February 1, 2026. Interest is a scale using values from 0-100. Summary statistics for the full time series reveal a mean of 43.97, median of 32.50, and a range from 20 to 100.

Figure 1 shows the time series plot of the original data values. Figure 1 displays a clear right-skew and a peak in trend before gradual decline begins around mid-2022. This behavior motivates our two-period trend analyses. Figure 1 also reveals that this data is not stationary and has unstable variance.

The scatter plot in Figure 2 displays the January 2026 outlier that possibly corresponds to the heightened anticipation surrounding BTS members after they completed their military service following a four year hiatus. This outlier could represent a genuine spike in interest and will be further analyzed through parallel forecasting measures.

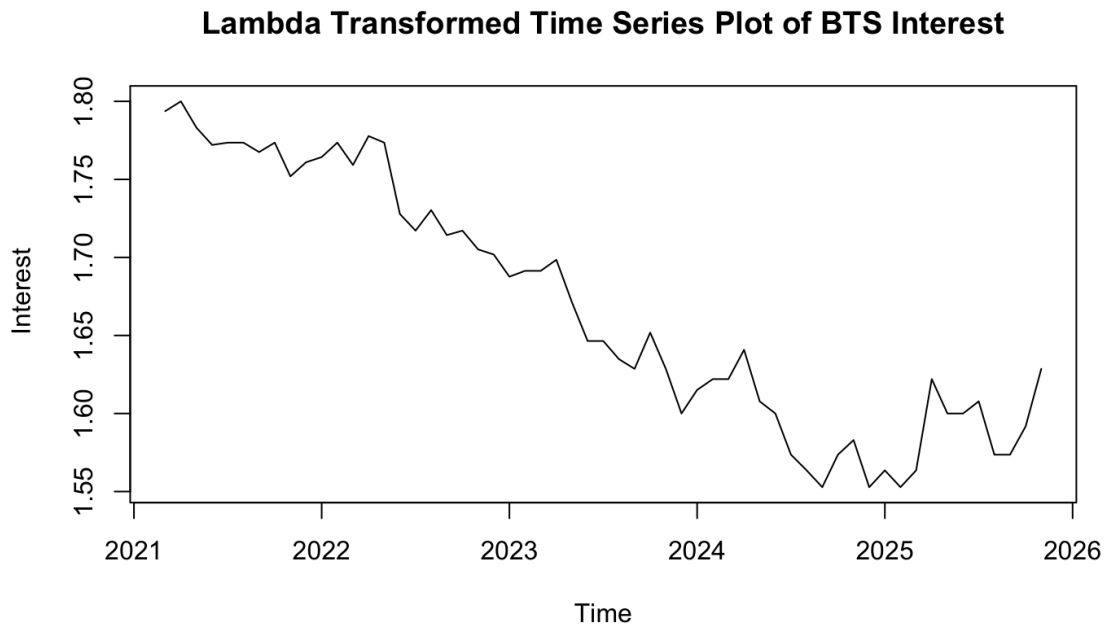

Figure 3: Time Series After Lambda-Transformation

4. Data analysis

To prepare the BTS time series for analysis, a Box–Cox (lambda) transformation was applied to stabilize the variance with $\lambda = -0.5$. Figure 3 shows the transformed series and improvements in skew and variance stability. After transformation, the seasonal component was estimated using monthly averages and then removed from the series to obtain a deseasonalized time series (Figure 4). Removing seasonality first allows the long-term trend to be estimated more clearly.

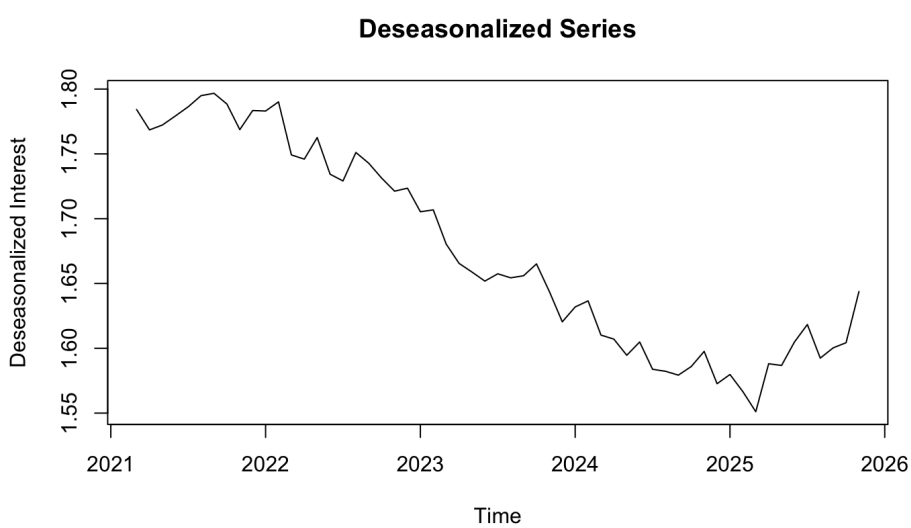

Figure 4: Time Series Plot after Removing Seasonality and Lambda Transformation

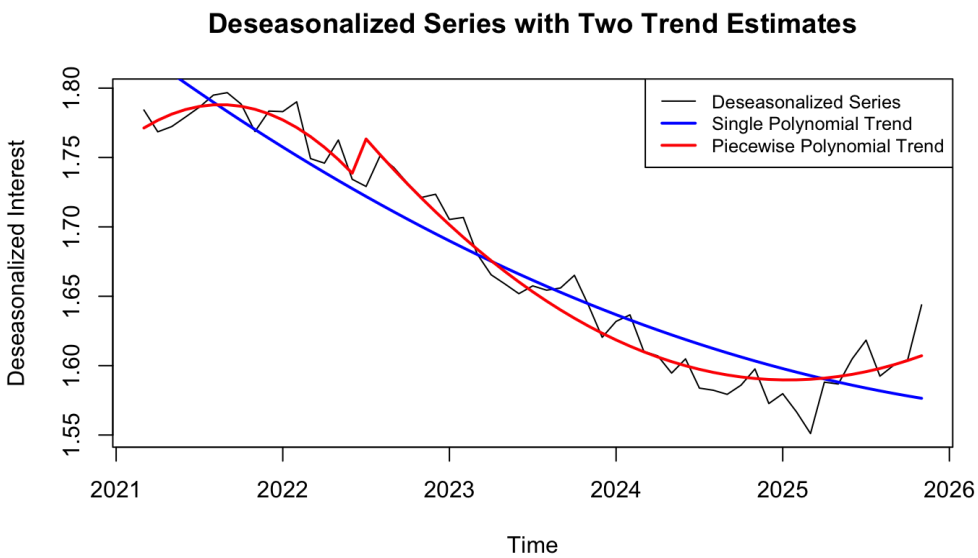

Figure 5: Single vs Piecewise Polynomial Trend Estimation

Two trend estimation approaches were applied to the deseasonalized transformed series (Figure 5). First, a quadratic polynomial regression was fitted across the entire series to produce a single smooth trend estimate. Second, a piecewise regression was applied by splitting the series into two windows representing the early high-interest period and the later declining period beginning in mid-2022. This piecewise approach allows the model to capture potential structural changes in the long-term trend that may not be well represented by a single global polynomial.

Observation of the ACF and PACF of the residuals after deseasonalizing and removing the single polynomial trend estimate Figure 7 shows non-stationary behavior. On the contrary, the ACF and PACF after deseasonalizing and removing the piecewise trend estimate in Figure 8 show stationary residual behavior. These observations suggest that the piecewise trend estimate is better for detrending and further ARMA model fitting.

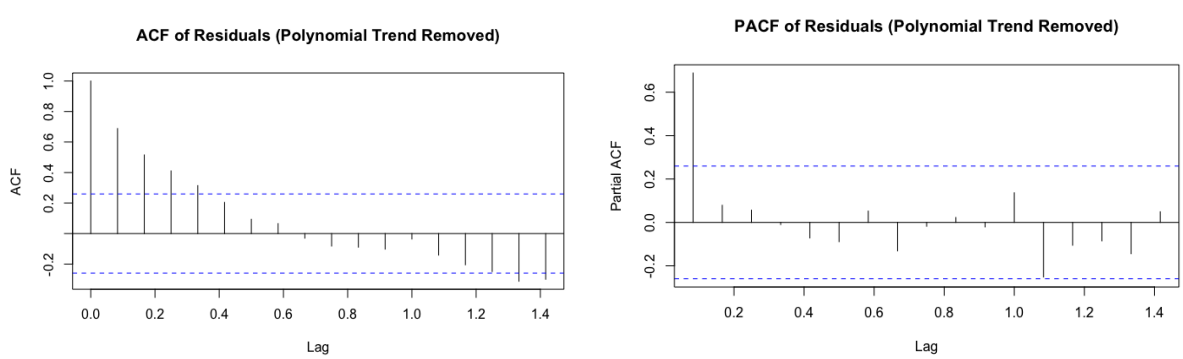

Figure 7: ACF and PACF of Residuals after Deseasonalizing and Polynomial Trend Removal

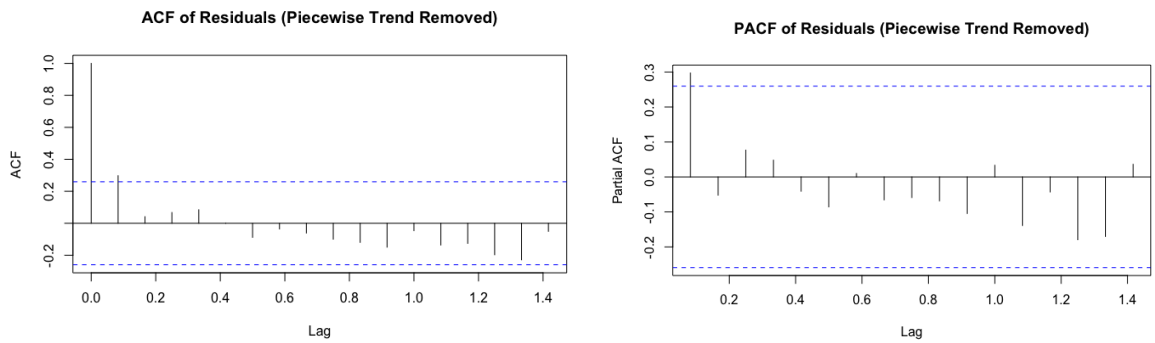

Figure 8: ACF and PACF of Residuals after Deseasonalizing and Piecewise Trend Removal

Plausible ARMA models including AR(1), MA(1), and ARMA(1,1) were fitted to the residual series with piecewise detrending using the R function Arima(). The model with the lowest AIC criteria was ARMA(1,1), so we decided to use that as our final model. After deseasonalizing and removing the piecewise trend estimate we fit an ARMA(1,1) model. Figure 8 displays residual white noise with all lags within the confidence bounds. This suggests our chosen model and removal of the piecewise trend estimate proficiently captures the remaining dependence structure in the residual series.

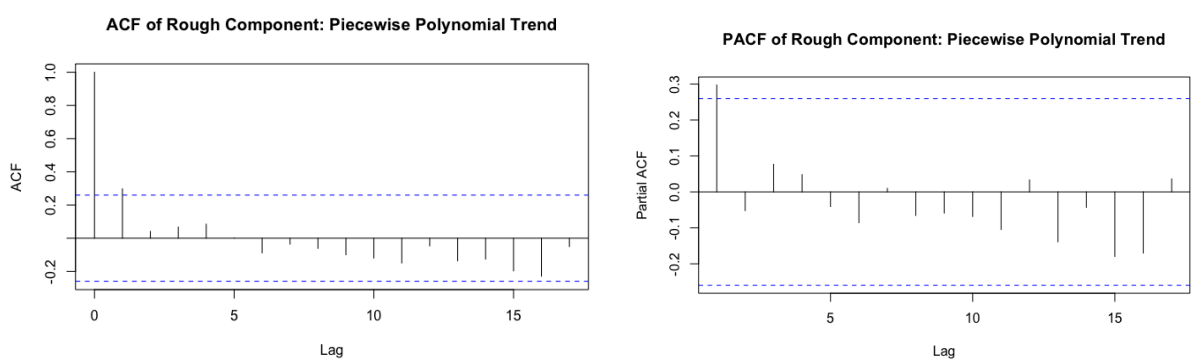

Figure 9: ACF and PACF of Fitted ARMA(1,1) Residuals

Forecasts were generated by projecting each component of the time series separately. The trend component was forecasted using the fitted quadratic regression models. The seasonal component was extended by repeating the estimated monthly seasonal pattern. Forecasts of the rough component were obtained from the fitted ARMA(1,1) model using the predict() function in R.

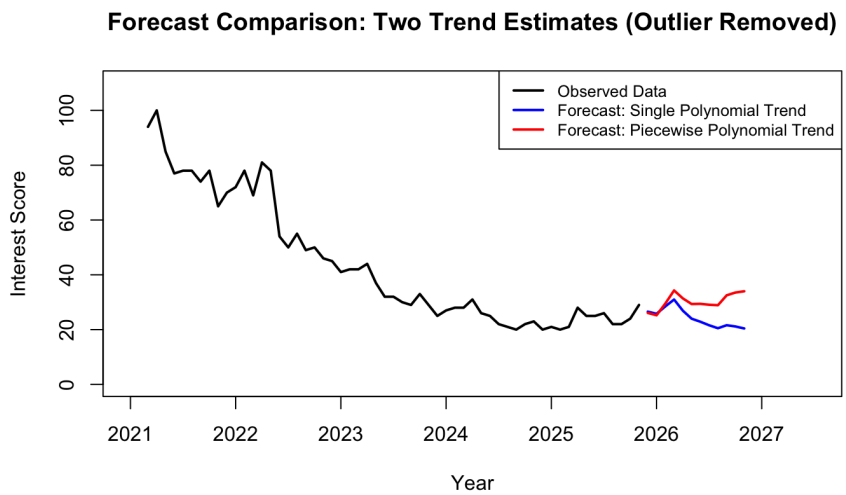

Figure 10: Forecasting using Piecewise vs Polynomial Trend Estimate

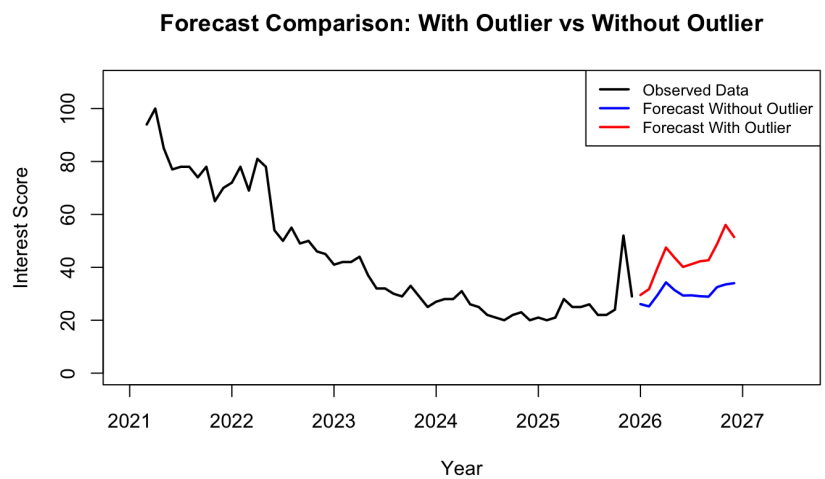

Figure 11: Forecasting With vs Without the Outlier

Figure 11 shows the forecast obtained after removing the January 2026 outlier. The predicted values follow the existing downward trend with moderate seasonal fluctuations. The same procedure was then applied to the full dataset, including the outlier in Figure. Although the forecast begins at a slightly higher level due to the spike, the overall downward trajectory remains similar across both analyses.

5. Discussion

The goal of this analysis was to evaluate how the January 2026 outlier and polynomial versus piecewise trend estimation and removal influences forecasting of BTS search interest. We found that the piecewise trend produces more stationary residuals, and therefore provides a more appropriate detrending method for ARMA modeling.

Based on AIC, the ARMA(1,1) model provided the best fit, and diagnostic plots indicated that the remaining residuals behave approximately like white noise.

The forecasting comparisons illustrate the impact of both trend modeling and the January 2026 outlier. Forecasts based on the piecewise trend remain higher than those from the single polynomial trend because the piecewise model captures the recent stabilization in search interest rather than continuing the overall long-term decline. When the outlier is removed, forecasts continue the gradual downward pattern observed in recent years. When the outlier is included, forecasts begin at a higher level but quickly return to a similar downward trend.

6. Conclusion

Using the piecewise polynomial trend estimate for detrending produced more stationary residuals, whereas the single polynomial trend estimate did not fully remove the dependence in the series. Forecast comparisons indicate that the January 2026 outlier increases the short-term forecast level but does not substantially change the overall downward trajectory of BTS search interest. If the January 2026 observation reflects genuine anticipation surrounding BTS's comeback, the forecasts suggest that public attention may experience a temporary increase before returning to the broader declining trend.

7. References

BTS time series data:
https://trends.google.com/trends/explore?date=today%205-y&q=bts&hl=en-US

8. Appendix

```{r}
library(forecast)
library(ggplot2)
# Data Acquisition
# import data file
df <- read.csv("multiTimeline.csv", skip = 2, header = TRUE)
# rename columns
colnames(df) <- c("Time", "Interest")
# converts time to date
df$Time <- as.Date(df$Time)
# display the first few rows
head(df)
# interest summary
summary(df$Interest)
# data with outlier removed
df_full  <- df
df_clean <- df[df$Time != as.Date("2026-01-01"), ]
```

```
# convert data to time series objects
bts_ts_full  <- ts(df_full$Interest, frequency = 12, start = c(2021, 3))
bts_ts_clean <- ts(df_clean$Interest, frequency = 12, start = c(2021, 3))
```
```{r}
# Data Visualization
# scatter plot of original time series
ggplot(df_full, aes(x = Time, y = Interest)) +
  geom_point(color = "steelblue", size = 2, alpha = 0.7) +
  scale_x_date(date_breaks = "6 months", date_labels = "%b %Y") +
  labs(title = "Monthly Google Search Interest for BTS",
       x = "Time", y = "Interest") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
# histogram showing the distribution of Interest
hist(df_full$Interest,
     main = "Histogram of BTS Search Interest",
     xlab = "Interest")
# shows significant skew
# convert data to time series object
bts_ts <- ts(df$Interest, frequency = 12, start = c(2021,3))
# time series plot of original BTS data
plot(bts_ts,
     main = "Time Series Plot of BTS Interest",
     ylab = " Interest",
     xlab = "Time")
```
```{r}
# Data Preparation
# check for missing values
sum(is.na(df_clean$Interest))   # should be 1 because Jan 2026 is replaced by NA
# boxcox lambda approximation for variance stability
lambda_clean <- BoxCox.lambda(na.omit(bts_ts_clean))
lambda_clean
# apply lambda transformation to clean series
bts_bc_clean <- ts(BoxCox(as.numeric(bts_ts_clean), lambda = lambda_clean),
                   start = c(2021, 3), frequency = 12)
# histogram after lambda transformation
hist(na.omit(bts_bc_clean),
     main = "Histogram of Lambda-Transformed BTS Interest",
     xlab = "Transformed Interest")
# time series plot after lambda transformation
plot(bts_bc_clean,
     main = "Lambda Transformed Time Series Plot of BTS Interest",
```

```
    ylab = "Transformed Interest",
    xlab = "Time")
# Deseasonalize
# month index
month_id_clean <- cycle(bts_bc_clean)
# estimate monthly seasonal means
mu_clean <- array(0, 12)
for(k in 1:12){
  mu_clean[k] <- mean(bts_bc_clean[month_id_clean == k], na.rm = TRUE)
}
# center so that seasonal effects sum to zero
mu_clean <- mu_clean - mean(mu_clean)
# create seasonal component
seasonal_component_clean <- ts(mu_clean[month_id_clean],
                               start = c(2021, 3), frequency = 12)
# deseasonalized series
deseasonal_clean <- bts_bc_clean - seasonal_component_clean
# plot estimated seasonal component
plot(seasonal_component_clean,
    main = "Estimated Seasonal Component",
    ylab = "Seasonality",
    xlab = "Time")
# plot deseasonalized series
plot(deseasonal_clean,
    main = "Deseasonalized Series",
    ylab = "Deseasonalized Interest",
    xlab = "Time")
abline(h = 0, col = "red", lty = 2)
```
```{r}
# Analyzing the “smooth” component:
# time index for full clean series
t_clean <- 1:length(deseasonal_clean)
# trend estimate fitting to one polynomial
trend_fit_poly_clean <- lm(as.numeric(deseasonal_clean) ~ t_clean + I(t_clean^2),
                           na.action = na.exclude)
trend_poly_clean <- ts(fitted(trend_fit_poly_clean),
                       start = c(2021, 3), frequency = 12)
# trend estimate using piecewise polynomia fit
deseasonal_early <- window(deseasonal_clean, end = c(2022, 6))
deseasonal_late  <- window(deseasonal_clean, start = c(2022, 7))
t_early <- 1:length(deseasonal_early)
t_late  <- 1:length(deseasonal_late)
trend_fit_early <- lm(as.numeric(deseasonal_early) ~ t_early + I(t_early^2),
```

```
                    na.action = na.exclude)
trend_fit_late  <- lm(as.numeric(deseasonal_late) ~ t_late + I(t_late^2),
                    na.action = na.exclude)
trend_piece_early <- ts(fitted(trend_fit_early),
                     start = start(deseasonal_early), frequency = 12)
trend_piece_late  <- ts(fitted(trend_fit_late),
                     start = start(deseasonal_late), frequency = 12)
# combine early and late fitted trends into one piecewise trend series
trend_piece_clean <- ts(rep(NA, length(deseasonal_clean)),
                     start = c(2021, 3), frequency = 12)
window(trend_piece_clean, end = c(2022, 6)) <- trend_piece_early
window(trend_piece_clean, start = c(2022, 7)) <- trend_piece_late
# plot both trend estimates
plot(deseasonal_clean,
    main = "Deseasonalized Series with Two Trend Estimates",
    ylab = "Deseasonalized Interest",
    xlab = "Time")
lines(trend_poly_clean, col = "blue", lwd = 2)
lines(trend_piece_clean, col = "red", lwd = 2)
legend("topright",
      legend = c("Deseasonalized Series", "Single Polynomial Trend", "Piecewise Polynomial Trend"),
      col = c("black", "blue", "red"),
      lty = c(1, 1, 1),
      lwd = c(1, 2, 2),
      cex = 0.8)
# compare fitted trend errors
mse_poly_clean <- mean((deseasonal_clean - trend_poly_clean)^2, na.rm = TRUE)
mse_piece_clean <- mean((deseasonal_clean - trend_piece_clean)^2, na.rm = TRUE)
print(data.frame(
  Trend_Model = c("Single Polynomial", "Piecewise Polynomial"),
  MSE = c(mse_poly_clean, mse_piece_clean)
))
```

```{r}
# Analyzing the “rough” component:
rough_poly_clean  <- deseasonal_clean - trend_poly_clean
rough_piece_clean <- deseasonal_clean - trend_piece_clean
# ACF/PACF for polynomial trend residuals
acf(na.omit(as.numeric(rough_poly_clean)),
   main = "ACF of Rough Component: Single Polynomial Trend")
pacf(na.omit(as.numeric(rough_poly_clean)),
    main = "PACF of Rough Component: Single Polynomial Trend")

# ACF/PACF for piecewise trend residuals
```

```
acf(na.omit(as.numeric(rough_piece_clean)),
   main = "ACF of Rough Component: Piecewise Polynomial Trend")
pacf(na.omit(as.numeric(rough_piece_clean)),
    main = "PACF of Rough Component: Piecewise Polynomial Trend")
# Single polynomial trend residuals
fit_ar1_poly   <- arima(na.omit(rough_poly_clean),  order = c(1,0,0))
fit_ma1_poly   <- arima(na.omit(rough_poly_clean),  order = c(0,0,1))
fit_arma11_poly <- arima(na.omit(rough_poly_clean), order = c(1,0,1))
# Piecewise polynomial trend residuals
fit_ar1_piece   <- arima(na.omit(rough_piece_clean),  order = c(1,0,0))
fit_ma1_piece   <- arima(na.omit(rough_piece_clean),  order = c(0,0,1))
fit_arma11_piece <- arima(na.omit(rough_piece_clean), order = c(1,0,1))
# AIC comparison tables
aic_poly <- c(fit_ar1_poly$aic, fit_ma1_poly$aic, fit_arma11_poly$aic)
aic_piece <- c(fit_ar1_piece$aic, fit_ma1_piece$aic, fit_arma11_piece$aic)
print(data.frame(
  Model = c("AR(1)", "MA(1)", "ARMA(1,1)"),
  AIC_Polynomial = aic_poly,
  AIC_Piecewise = aic_piece
))
```
```{r}
# Forecasting the two trend estimates
h <- 12
# future indices
future_t_clean <- (length(deseasonal_clean) + 1):(length(deseasonal_clean) + h)
future_t_late  <- (length(deseasonal_late) + 1):(length(deseasonal_late) + h)
# single polynomial trend estimate
trend_forecast_poly_clean <- predict(trend_fit_poly_clean,
                              newdata = data.frame(t_clean = future_t_clean))
# seasonality forecast
seasonal_forecast_clean <- rep(mu_clean, length.out = h)
# choose ARMA(1,1) for now to match your earlier structure
rough_forecast_poly_obj <- predict(fit_arma11_poly, n.ahead = h)
rough_forecast_poly <- rough_forecast_poly_obj$pred
rough_se_poly <- rough_forecast_poly_obj$se
# combine
final_bc_poly_clean <- trend_forecast_poly_clean +
                    seasonal_forecast_clean +
                    rough_forecast_poly
final_forecast_poly_clean <- InvBoxCox(final_bc_poly_clean, lambda = lambda_clean)
# forecast using piecewise polynomial
trend_forecast_piece_clean <- predict(trend_fit_late,
                               newdata = data.frame(t_late = future_t_late))
```

```
rough_forecast_piece_obj <- predict(fit_arma11_piece, n.ahead = h)
rough_forecast_piece <- rough_forecast_piece_obj$pred
rough_se_piece <- rough_forecast_piece_obj$se
# combine
final_bc_piece_clean <- trend_forecast_piece_clean +
                    seasonal_forecast_clean +
                    rough_forecast_piece
final_forecast_piece_clean <- InvBoxCox(final_bc_piece_clean, lambda = lambda_clean)
forecast_poly_ts_clean <- ts(final_forecast_poly_clean,
                         start = tsp(bts_ts_clean)[2] + 1/12,
                         frequency = 12)
forecast_piece_ts_clean <- ts(final_forecast_piece_clean,
                          start = tsp(bts_ts_clean)[2] + 1/12,
                          frequency = 12)
# convert to ts objects
lines(forecast_poly_ts_clean, col = "blue", lwd = 2)
lines(forecast_piece_ts_clean, col = "red", lwd = 2)
# plot the two clean-data forecast methods
plot(bts_ts_clean,
    xlim = c(2021, 2027.5),
    ylim = c(0, max(bts_ts_full, final_forecast_poly_clean, final_forecast_piece_clean, na.rm = TRUE) +
10),
    main = "Forecast Comparison: Two Trend Estimates (Outlier Removed)",
    ylab = "Interest Score",
    xlab = "Year",
    lwd = 2)
lines(forecast_poly_ts_clean, col = "blue", lwd = 2)
lines(forecast_piece_ts_clean, col = "red", lwd = 2)
legend("topright",
      legend = c("Observed Data", "Forecast: Single Polynomial Trend", "Forecast: Piecewise Polynomial
Trend"),
      col = c("black", "blue", "red"),
      lty = c(1, 1, 1),
      lwd = c(2, 2, 2),
      cex = 0.8)
```
```{r}
# forecast with outlier
lambda_full <- BoxCox.lambda(bts_ts_full)
bts_bc_full <- ts(BoxCox(as.numeric(bts_ts_full), lambda = lambda_full),
                start = c(2021, 3), frequency = 12)
# estimate seasonality for full series
month_id_full <- cycle(bts_bc_full)
mu_full <- array(0, 12)
```

```
for(k in 1:12){
  mu_full[k] <- mean(bts_bc_full[month_id_full == k], na.rm = TRUE)
}
mu_full <- mu_full - mean(mu_full)

seasonal_component_full <- ts(mu_full[month_id_full],
                              start = c(2021, 3), frequency = 12)
# deseasonalize
deseasonal_full <- bts_bc_full - seasonal_component_full
# piecewise trend on full data
deseasonal_early_full <- window(deseasonal_full, end = c(2022, 6))
deseasonal_late_full  <- window(deseasonal_full, start = c(2022, 7))
t_early_full <- 1:length(deseasonal_early_full)
t_late_full  <- 1:length(deseasonal_late_full)
trend_fit_early_full <- lm(as.numeric(deseasonal_early_full) ~ t_early_full + I(t_early_full^2),
                           na.action = na.exclude)
trend_fit_late_full <- lm(as.numeric(deseasonal_late_full) ~ t_late_full + I(t_late_full^2),
                          na.action = na.exclude)
trend_piece_early_full <- ts(fitted(trend_fit_early_full),
                             start = start(deseasonal_early_full), frequency = 12)
trend_piece_late_full <- ts(fitted(trend_fit_late_full),
                            start = start(deseasonal_late_full), frequency = 12)
trend_piece_full <- ts(rep(NA, length(deseasonal_full)),
                       start = c(2021, 3), frequency = 12)
window(trend_piece_full, end = c(2022, 6)) <- trend_piece_early_full
window(trend_piece_full, start = c(2022, 7)) <- trend_piece_late_full
# rough component
rough_piece_full <- deseasonal_full - trend_piece_full
# ARMA fit
fit_arma11_full <- arima(na.omit(rough_piece_full), order = c(1,0,1))
# forecasts
future_t_late_full <- (length(deseasonal_late_full) + 1):(length(deseasonal_late_full) + h)
trend_forecast_full <- predict(trend_fit_late_full,
                               newdata = data.frame(t_late_full = future_t_late_full))
seasonal_forecast_full <- rep(mu_full, length.out = h)
rough_forecast_full_obj <- predict(fit_arma11_full, n.ahead = h)
rough_forecast_full <- rough_forecast_full_obj$pred
final_bc_full <- trend_forecast_full +
                 seasonal_forecast_full +
                 rough_forecast_full
final_forecast_full <- InvBoxCox(final_bc_full, lambda = lambda_full)
forecast_full_ts <- ts(final_forecast_full, start = c(2026, 3), frequency = 12)
```
```{r}
```

```
# final plot: with outlier vs without outlier
forecast_full_ts <- ts(final_forecast_full,
                       start = tsp(bts_ts_full)[2] + 1/12,
                       frequency = 12)
forecast_piece_ts_clean <- ts(final_forecast_piece_clean,
                              start = tsp(bts_ts_full)[2] + 1/12,
                              frequency = 12)
plot(bts_ts_full,
     xlim = c(2021, 2027.5),
     ylim = c(0, max(bts_ts_full,
                     final_forecast_piece_clean,
                     final_forecast_full,
                     na.rm = TRUE) + 10),
     main = "Forecast Comparison: With Outlier vs Without Outlier",
     ylab = "Interest Score",
     xlab = "Year",
     lwd = 2)
lines(forecast_piece_ts_clean, col = "blue", lwd = 2)
lines(forecast_full_ts, col = "red", lwd = 2)
legend("topright",
       legend = c("Observed Data", "Forecast Without Outlier", "Forecast With Outlier"),
       col = c("black", "blue", "red"),
       lty = c(1, 1, 1),
       lwd = c(2, 2, 2),
       cex = 0.8)
print(data.frame(
  Month = seq(as.Date("2026-03-01"), by = "month", length.out = h),
  Forecast_Without_Outlier = as.numeric(final_forecast_piece_clean),
  Forecast_With_Outlier = as.numeric(final_forecast_full)
))
```